FALL BREAK 2026 · PARALLEL PROGRAMMING & GPU BOOTCAMP

# Numerical Methods for High-Performance Computing

**Mansi Bhati**

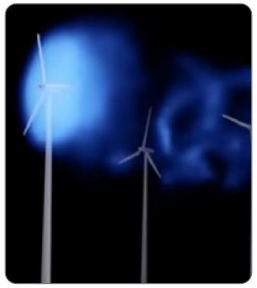
Department of Chemistry, Princeton University

WHY THIS MATTERS

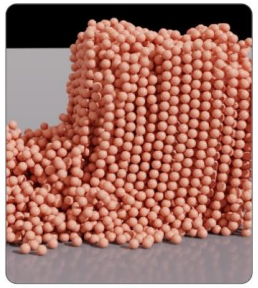
# Linear algebra runs under almost everything



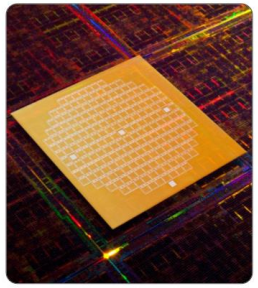
**cuDSS**
CAE



**PhysicsNeMo**
AI Physics



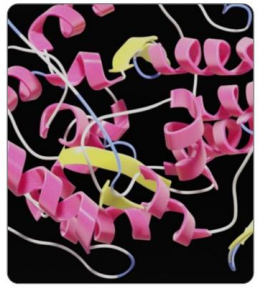
**Warp**
Physical Simulation



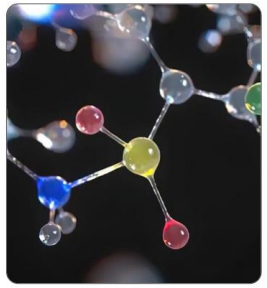
**cuLitho**
Computational Lithography



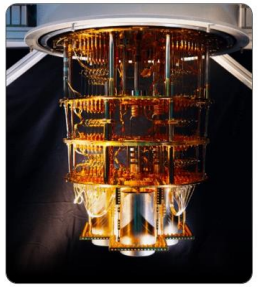
**ALCHEMI**
AI Materials Science



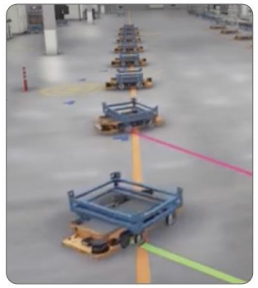
**cuEquivariance**
Drug & Materials Discovery



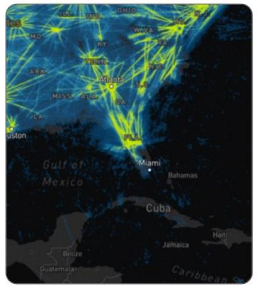
**Parabricks**
Gene Sequencing



**CUDA-Q**
Quantum Computing



**cuOpt**
Decision Optimization



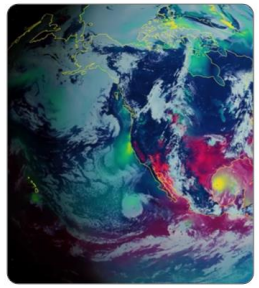

**cuDF**
Data Processing



**cuPyNumeric**
Numerical Computing



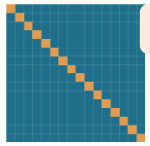
**Holoscan**
Edge HPC



**Earth-2**
Weather Analytics

THE LANDSCAPE

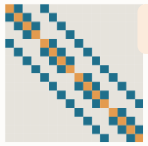
# Seven patterns cover most of scientific computing



TODAY'S FOCUS

## 1 Dense linear algebra

Every unknown interacts with every other: store all $n^2$ numbers.



TODAY'S FOCUS

## 2 Sparse linear algebra

Each unknown touches a few neighbours: store only the nonzeros.



## 3 Spectral (FFT)

Switch a signal into its frequencies.



## 4 N-body

Particles that all pull on each other.



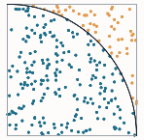
## 5 Structured grids

Each point on a regular grid updated from its neighbours.



## 6 Unstructured grids

The same on an irregular mesh shaped to an object.



## 7 Monte Carlo

Many independent random trials.

### Why focus on 1 and 2?

Solving equations on grids (5, 6) produces sparse linear algebra. N-body and spectral codes call linear algebra too.

After Asanovic et al., "The Landscape of Parallel Computing Research: A View from Berkeley," UC Berkeley, 2006.

THE BIG PICTURE

# Linear algebra asks one of three questions




## Multiply

**C = A B: combine data**

- Neural-network layers: inputs × learned weights, most of the computing in AI
- One step of a simulation: each value updated from its neighbours




## Solve

**A x = b: know the effect b, find the cause x**

- Heat or airflow on a mesh; stress in a structure
- Currents in a circuit
- Fitting a model to data




## Eigenvalues

**A x = λ x: find the natural modes**

- Vibrations of a bridge
- Google's PageRank
- Energy levels of a molecule; main patterns in data (PCA)

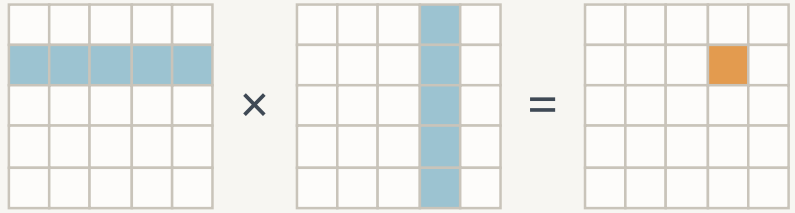
Each can come with a dense or a sparse and size of the matrix changes the algorithm

Diagram after S. Huber, "Numerical Methods for HPC," IHPCSS 2026.

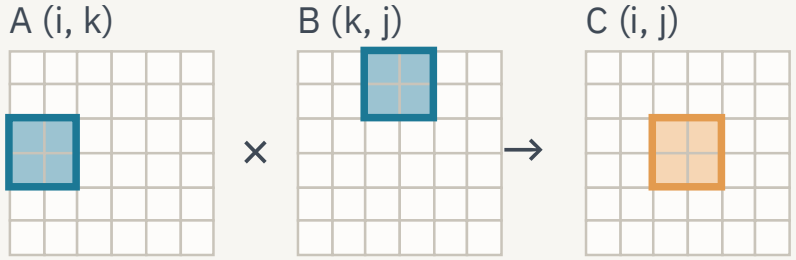
DENSE LINEAR ALGEBRA · THE COST OF MOVING DATA

# Arithmetic is cheap; moving data is expensive

## Matrix multiply is the exception



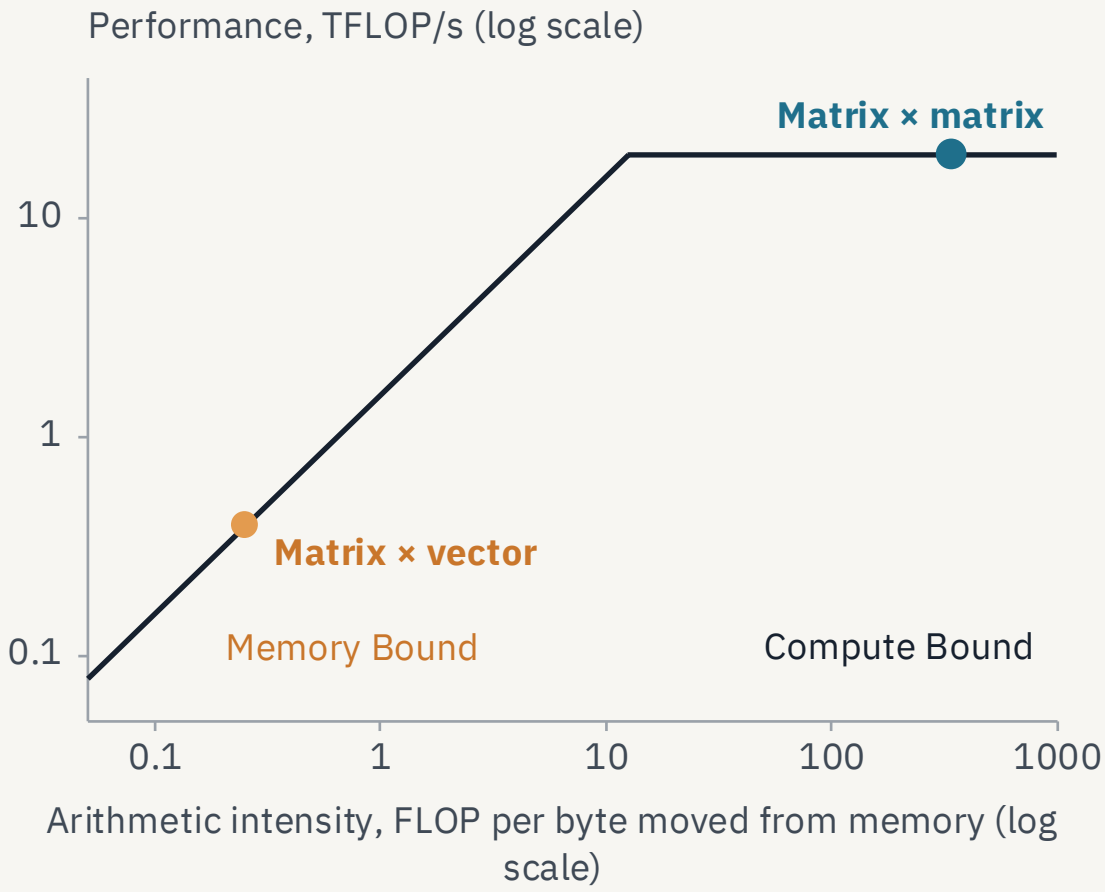
Row of A × column of B = one entry of C




Tile of A × tile of B → add to tile of C; repeat over k

Load tiles once. Reuse before fetching more.

CPUs: Intel MKL and OpenBLAS
NVIDIA GPUs: cuBLAS




**Reuse data → move fewer bytes → approach peak FLOP/s**

DENSE LINEAR ALGEBRA · SOLVING $Ax = b$

# LU factorization: from elimination to blocked GEMM

## 1 Mathematics: Gaussian elimination → LU

$Ax = b$ → $A = LU$ → $LUx = b$

Think from the inside out: define $y = Ux$.

1. Solve $Ly = b$ → 2. Solve $Ux = y$

Why easier? L and U are triangular, so each solve reveals the unknowns one at a time.



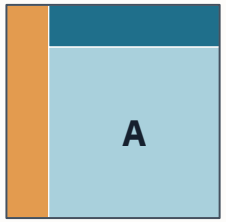
L



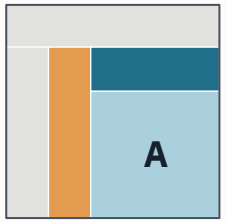
U

**Factor A once; reuse the same L and U for new right-hand sides.**

## 2 HPC: reorganize the same LU into blocks



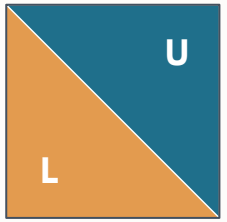

Step 1




Step 2




Step 3




Step 4

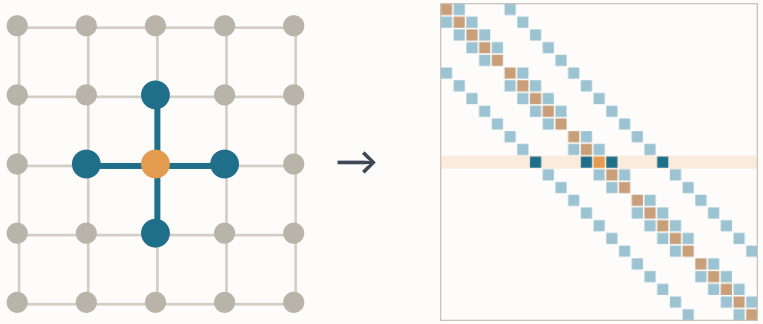
**Same LU factorization — but most of the work becomes GEMM, so the hardware can reuse data efficiently.**

CPU: LAPACK / MKL · NVIDIA GPU: cuSOLVER · Many nodes: ScaLAPACK / SLATE

SPARSE LINEAR ALGEBRA

# Sparse matrices: store only what is there

### Where they come from



Heat flow on a 5 × 5 grid: the centre point talks only to its 4 neighbours, so **its row (highlighted)** has 5 nonzeros. Every row stays this short, however big the grid.

### The pattern matters




Same number of nonzeros, very different behaviour: the pattern decides how regularly memory is read and how easily the work splits across cores.

### Store only the nonzeros

For each nonzero keep its value and its column, plus where each row starts. Most libraries use this "compressed sparse row" (CSR) layout.

Heat flow on a 1000 × 1000 grid (a million unknowns):

**64 MB** stored sparsely **8 TB** with every zero

But sparse does not automatically mean cheap: solving by elimination can fill in the zeros.

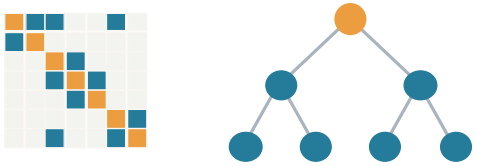
SPARSE LINEAR SYSTEMS · DIRECT METHODS

# A sparse direct solver does three different jobs

Analyze the structure → factor the numbers → solve for a particular right-hand side

**PHASE 1**

## Analyze + reorder



### Same equations, better elimination order

Use only the nonzero pattern. Reorder the unknowns to reduce fill-in and reveal independent work that can run in parallel.

**PHASE 2**

## Factorize



### Use the numerical values to build L and U

After reordering, related columns can be grouped into dense blocks, so much of the expensive update becomes matrix–matrix multiplication again.

**PHASE 3**

## Solve

$A = LU \Rightarrow$ **1. Solve $Ly = b$** → **2. Solve $Ux = y$**

**Same A, new b? Reuse L and U — repeat only Phase 3.**

Triangular systems recover the unknowns one at a time.

CPU: PARDISO · NVIDIA GPU: cuDSS · Many nodes: MUMPS

SPARSE LINEAR SYSTEMS · ITERATIVE METHODS

# Iterative methods improve a guess using A × x

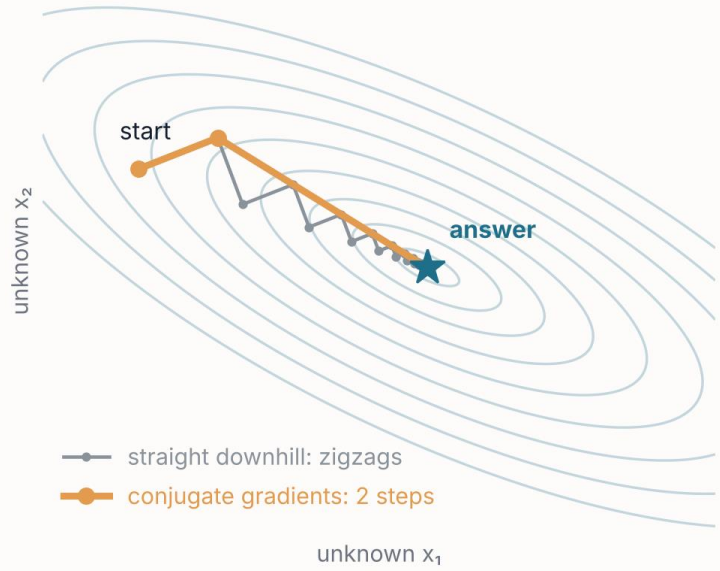
## The loop

1. Guess x.
2. Check how wrong it is: r = b − A x.
3. Use r to correct x.
4. Repeat until r is tiny.

## What makes it a Krylov method

$$\mathbf{r} \xrightarrow{\times A} \mathbf{Ar} \xrightarrow{\times A} \mathbf{A^2 r} \xrightarrow{\times A} \mathbf{A^3 r} \xrightarrow{\times A} \cdots$$

Each step can only multiply by A, so these are the only vectors you have. A Krylov method picks the **best combination** of them. CG does this for symmetric positive definite matrices; GMRES and BiCGStab for general ones.




Two unknowns: solving Ax = b is finding the bottom of a bowl. Conjugate gradients (CG) never undoes earlier progress, so it gets there in 2 steps.

No fill-in, no factorization: just multiply, combine, repeat.

SPARSE LINEAR SYSTEMS · ITERATIVE METHODS

# How many steps? It depends on matrix’ s geometry

## The condition number κ

**Small κ**
few steps



**Large κ**
many steps



κ compares the most and least A stretches a vector.

## Preconditioning: improve conditioning

Pick M close to A but cheap to undo, and solve $M^{-1}A\,x = M^{-1}b$ instead: same answer, better conditioning. A better M means fewer steps, but each step costs more.

## Jacobi



**M = diag(A)**

Scale each row independently

Low cost; highly parallel.
Often a weak preconditioner.

## Incomplete LU

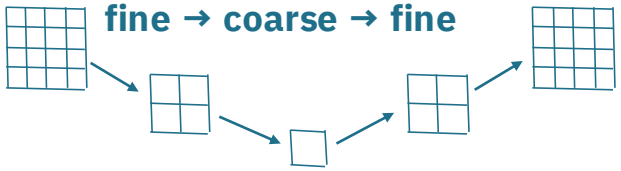
**$A \approx \tilde{L}\tilde{U}$ (selected fill-in dropped)**



Lower × upper sparse factors

Stronger approximation; triangular solves have dependencies.

## Multigrid




Smooth, correct on coarser grids, then return to the fine grid. Solving PDEs

**The best preconditioner is not always the strongest: setup cost, memory and CPU/GPU speed matter too**

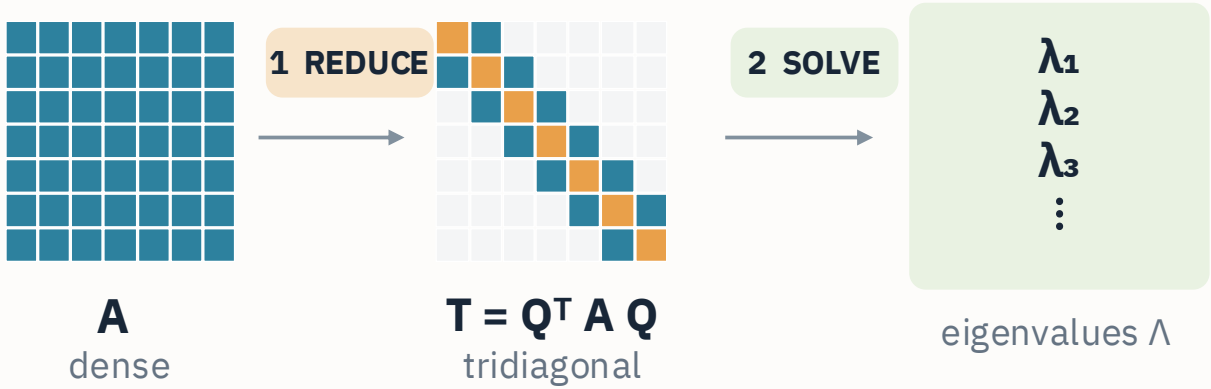
Libraries: hypre · PETSc · Trilinos · AmgX (NVIDIA) · Ginkgo

EIGENVALUE PROBLEMS · DENSE

# Need many eigenvalues? Reduce the problem first

For a dense symmetric matrix, simplify A without changing its eigenvalues.

## THE MATHEMATICAL WORKFLOW




**A**
dense

**T = Q^T A Q**
tridiagonal

eigenvalues Λ

### 1 Reduce

Householder reflections build an orthogonal Q so T = $Q^T$AQ. A and T have the same eigenvalues.

### 2 Solve

The tridiagonal problem is much cheaper. QR iteration or divide-and-conquer can finish the job.

**If eigenvectors are needed, transform them back with Q.**

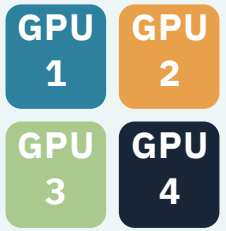
## WHY THIS MATTERS FOR HPC

### Block the reduction

Modern solvers reorganize the transformations so much of the expensive work becomes matrix–matrix updates.

**more data reuse** **closer to GEMM speed**

### Across multiple GPUs




Distribute matrix blocks across GPUs; applying the transformations needs communication.

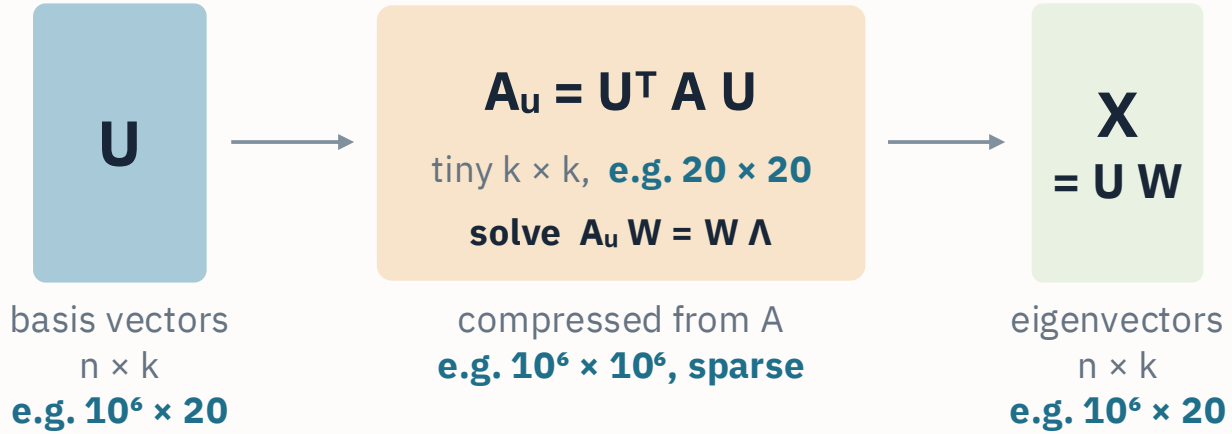
**Libraries: LAPACK · cuSOLVER · ELPA · SLATE**

EIGENVALUE PROBLEMS · LARGE / SPARSE

# Need only a few eigenvalues? Work in a small subspace

Instead of simplifying all of A, build a small space that captures the eigenvectors you care about.

## THE ITERATIVE IDEA




1 **Build** build U with repeated A × x

2 **Project** compress A to $U^T AU$

3 **Solve** solve the tiny eigenproblem

4 **Check** lift back; refine U if needed

*This projection-and-solve step is called Rayleigh–Ritz.*

**COMPUTATION**

Apply A to several vectors together for greater data reuse.

**COMMUNICATION**

Orthogonalizing vectors requires global reductions across GPUs.

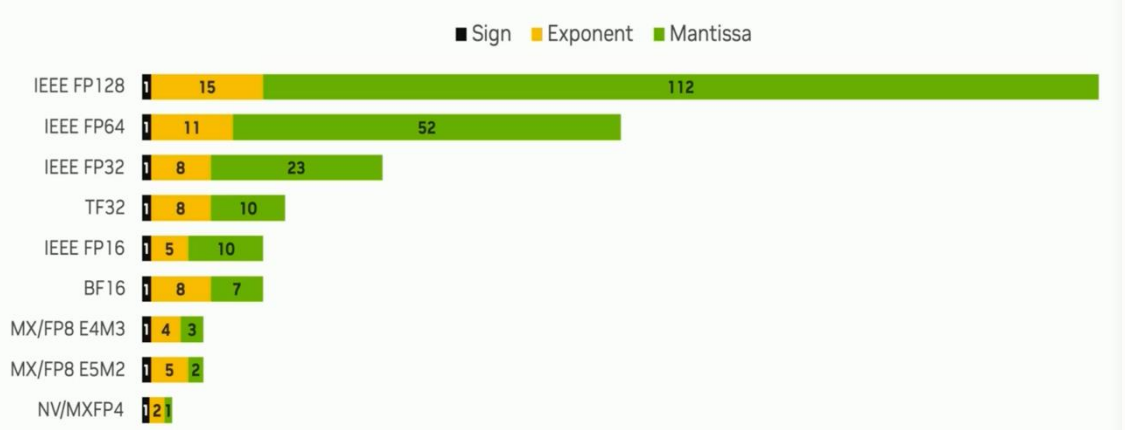
Examples: Lanczos · Davidson · LOBPCG — they mainly differ in how they build and improve U.

PRECISION

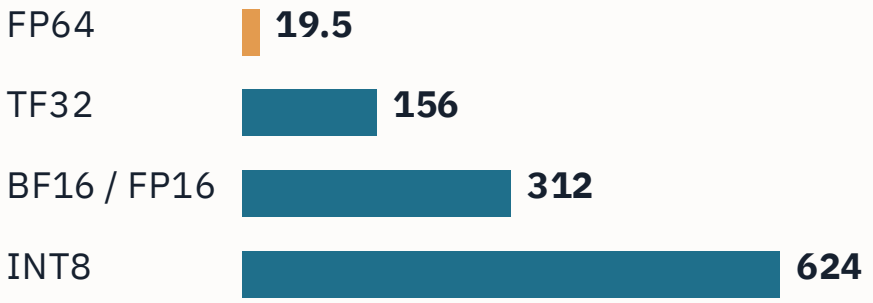
# Fewer digits, far more speed

## Fewer bits, fewer digits

$$x = (-1)^s \times 1.m \times 2^{e - \text{bias}}$$




## ...but far more speed (NVIDIA A100)




trillion operations per second on tensor cores: 32× more in INT8 than FP64. Also reduces memory traffic

**The catch:** fewer bits mean larger rounding errors, and ill-conditioned problems amplify them.

**Order matters:** in FP64, $(10^{16} + 1) - 10^{16} = 0$, so parallel sums can differ slightly from run to run.

Can we do most of the work in low precision and still get an FP64 answer? Two ways.

PRECISION · METHODS 1 AND 2

# Two routes to FP64 accuracy on low-precision hardware

## METHOD 1 · CHANGE THE ALGORITHM
### Mixed-precision iterative refinement

**1 Factorize in low precision**
Approximate solution from an FP16 LU

**2 Compute the residual in FP64**
r = b − Ax

**3 Correct the solution**
Reuse the low-precision factors

**4 Repeat until accurate enough**
FP64-level answer, subject to convergence

**Different steps run at different precisions**

## METHOD 2 · EMULATE FP64
### Ozaki scheme on tensor cores

**1 Split both FP64 inputs**
Break each number into scaled INT8 pieces

**2 Multiply the slice pairs**
Exact integer products on tensor cores

**3 Recombine in FP64**
Restore each scale and sum in FP64

**Rubin: 200 vs 33 TFLOP/s native FP64 (≈6×)**

Sources: NVIDIA technical blog, "Unlocking Tensor Core Performance with Floating Point Emulation in cuBLAS"; HPCwire, Feb 2026; The Register, Jan 2026.

WHAT TO USE TODAY

# Where to find these methods

| Problem | One node (CPU) | NVIDIA GPU | Many nodes (MPI) | From Python |
|---|---|---|---|---|
| Dense | LAPACK via MKL, OpenBLAS, AOCL | cuBLAS, cuSOLVER, MAGMA | ScaLAPACK, SLATE, cuBLASMp, cuSOLVERMp | NumPy, SciPy, CuPy |
| Sparse direct | PARDISO, SuiteSparse | cuDSS | MUMPS, SuperLU_DIST, STRUMPACK | scipy.sparse.linalg, nvmath-python |
| Iterative + preconditioners | PETSc, Trilinos, hypre, Ginkgo | AmgX; GPU builds of PETSc, hypre, Ginkgo | PETSc, Trilinos, hypre | SciPy, CuPy, petsc4py |
| Eigenvalues | LAPACK, ARPACK, PRIMME | cuSOLVER, MAGMA | ELPA, EigenExa, SLEPc, SLATE | scipy eigh / eigsh, slepc4py |

TO SUM UP

# Three things to take away

## 1

### Structure decides the method

What you are asking (multiply, solve, eigenvalues) and what the matrix looks like (dense, sparse, size) narrow the choice before any code.

## 2

### Data movement sets the speed

Matrix multiply reuses data and runs near peak, so dense algorithms turn their work into it. Sparse and iterative methods wait on memory and global sums.

## 3

### Precision is now a design choice

Do the heavy work in low precision; recover accuracy with refinement or emulation. κ tells you how many digits you can trust.

FOR YOUR OWN CODE, WALK THE CHAIN

**Problem** → **Method** → **Kernel** → **Hardware** → **Library**

**Efficient scientific computing needs both the numerical math and the hardware it runs on**

# Questions?

Thank you.

Mansi Bhati · mb3835@princeton.edu

FURTHER READING

# References


Saad. *Iterative Methods for Sparse Linear Systems*, 2nd ed. SIAM, 2003.

Trefethen & Bau. *Numerical Linear Algebra*. SIAM, 1997.

Davis, Rajamanickam & Sid-Lakhdar. A survey of direct methods for sparse linear systems. *Acta Numerica* 25, 2016.

Shewchuk. *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*. CMU, 1994.

Williams, Waterman & Patterson. Roofline: an insightful visual performance model. *CACM* 52(4), 2009.

Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM, 2002.

Carson & Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SISC* 40(2), 2018.

Haidar, Bayraktar, Tomov, Dongarra & Higham. Mixed-precision iterative refinement using tensor cores on GPUs. *Proc. R. Soc. A* 476, 2020.

Dongarra & Luszczek. HPL-MxP benchmark: mixed-precision algorithms and iterative refinement. *IJHPCA* 40(1), 2026.

Sustaining exascale performance on Aurora: HPL and HPL-MxP. arXiv:2604.09517, 2026.

NVIDIA. Unlocking Tensor Core performance with floating point emulation in cuBLAS. NVIDIA Technical Blog.

HPCwire. Genesis Mission will lean heavily on Ozaki scheme for FP64 capability. Feb 2026.

Ghysels & Vanroose. Hiding global synchronization latency in the preconditioned CG algorithm. *Parallel Computing* 40, 2014.

NVIDIA A100 and H100 datasheets · SuiteSparse Matrix Collection, sparse.tamu.edu